



UNIVERSITY
OF HULL

Concurrency

700105 Games Architecture & Concurrency

Introduction

- In this module we will consider
 - What is concurrency?
 - Threading
 - C++ 11/14/17/20/23
 - OpenMP
 - (Win32 / Python)
 - Concurrency control
 - Locks
 - Semaphores
 - Threading architectures
 - Lockless programming
 - Compute (GPGPU)
- What is a distributed system?
- Distributed Architectures
- Distributed Programming
 - Sockets

Concurrency

- Concurrency is:
 - “The illusion that two or more tasks are being performed in parallel.”
 - Achieved by an operating system kernel's support of multiple threads.
- Parallelism is:
 - “The reality that two or more tasks are being performed in parallel.”
 - Achieved by execution of tasks on multiple processors.

Common terms

- Multi-tasking
- Multi-core
- Multi-processor
- Multi-threaded
- Hyper-threaded
- Out of order execution

Why Concurrency?

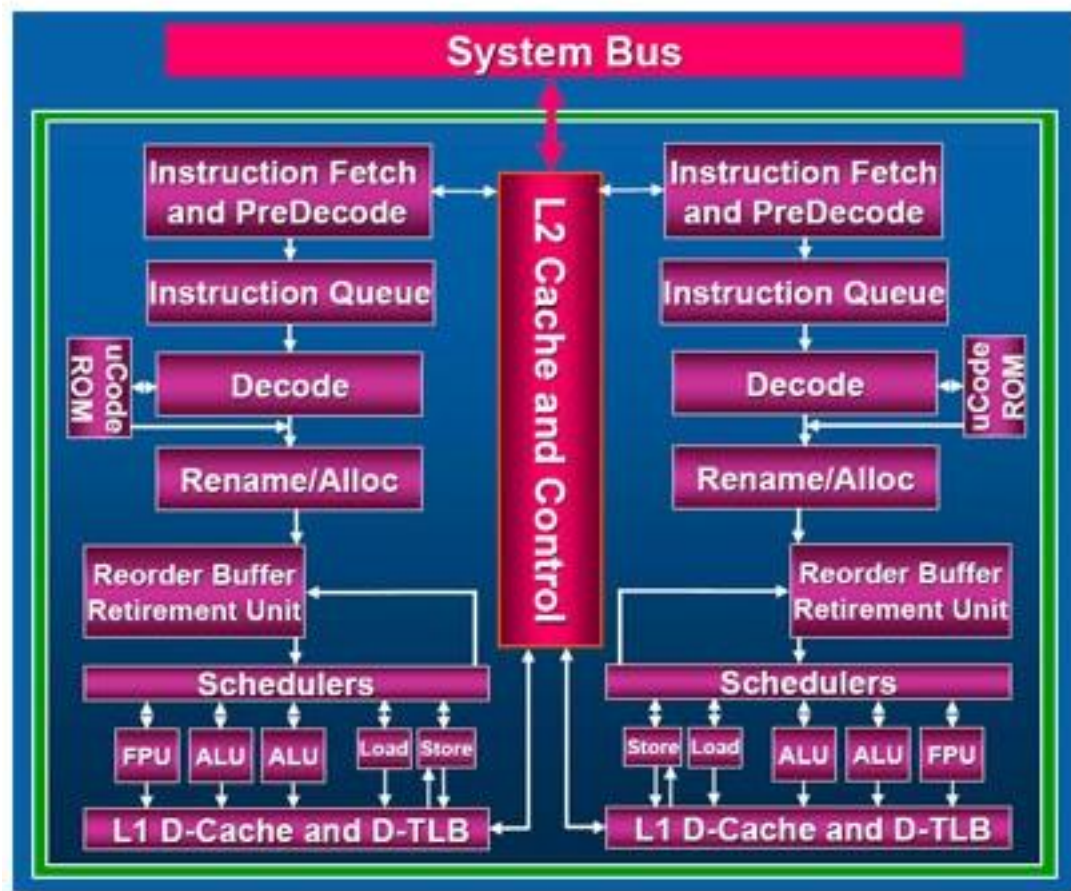
(Or how we got into this mess)

- Moores' Law predicts a doubling of processor performance every 18 months
- Processor design has reached physical limitations
 - Clock speeds have reached a limit around 3-4Ghz
 - Higher clock speeds generate more heat, require more power and have higher current leakage problems
- Hardware manufacturers have switched to multi-core (and/or hyper-threading / out of order execution)

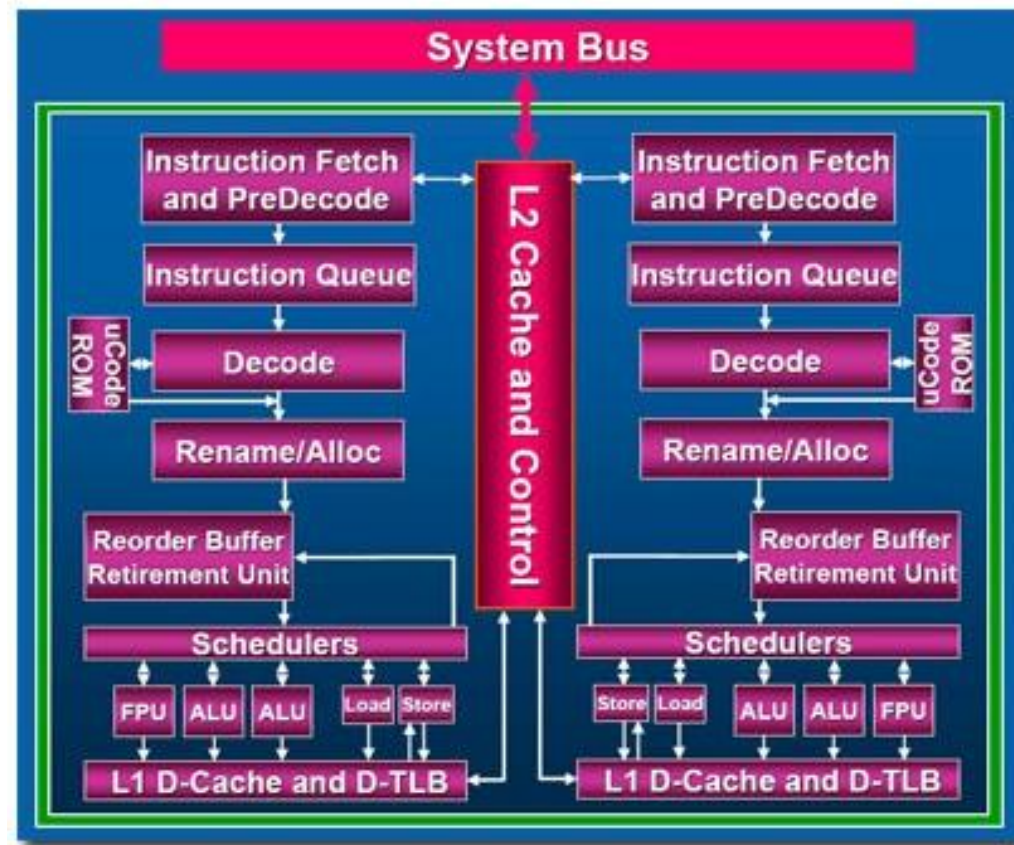
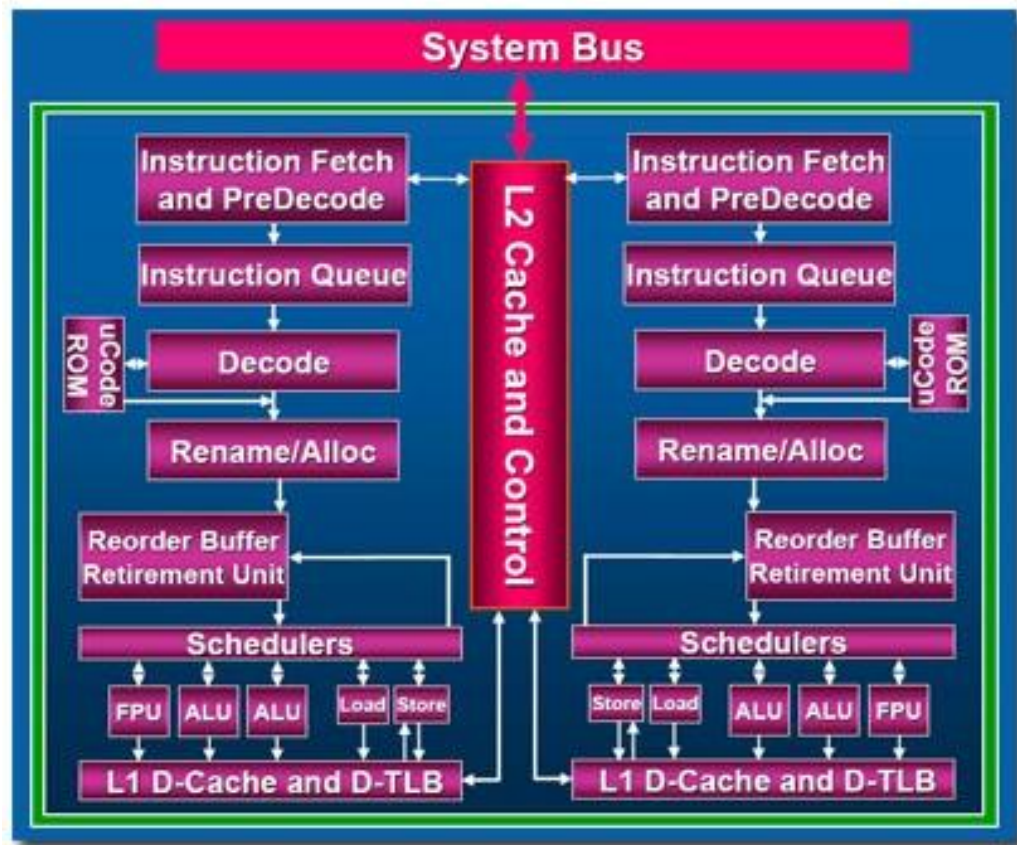
Multi-core vs Hyper-threading

- Hyper-threading
 - 10%-20% increase
- Multi-core (dual)
 - 60% , maybe
 - never 100% increase
 - Why?

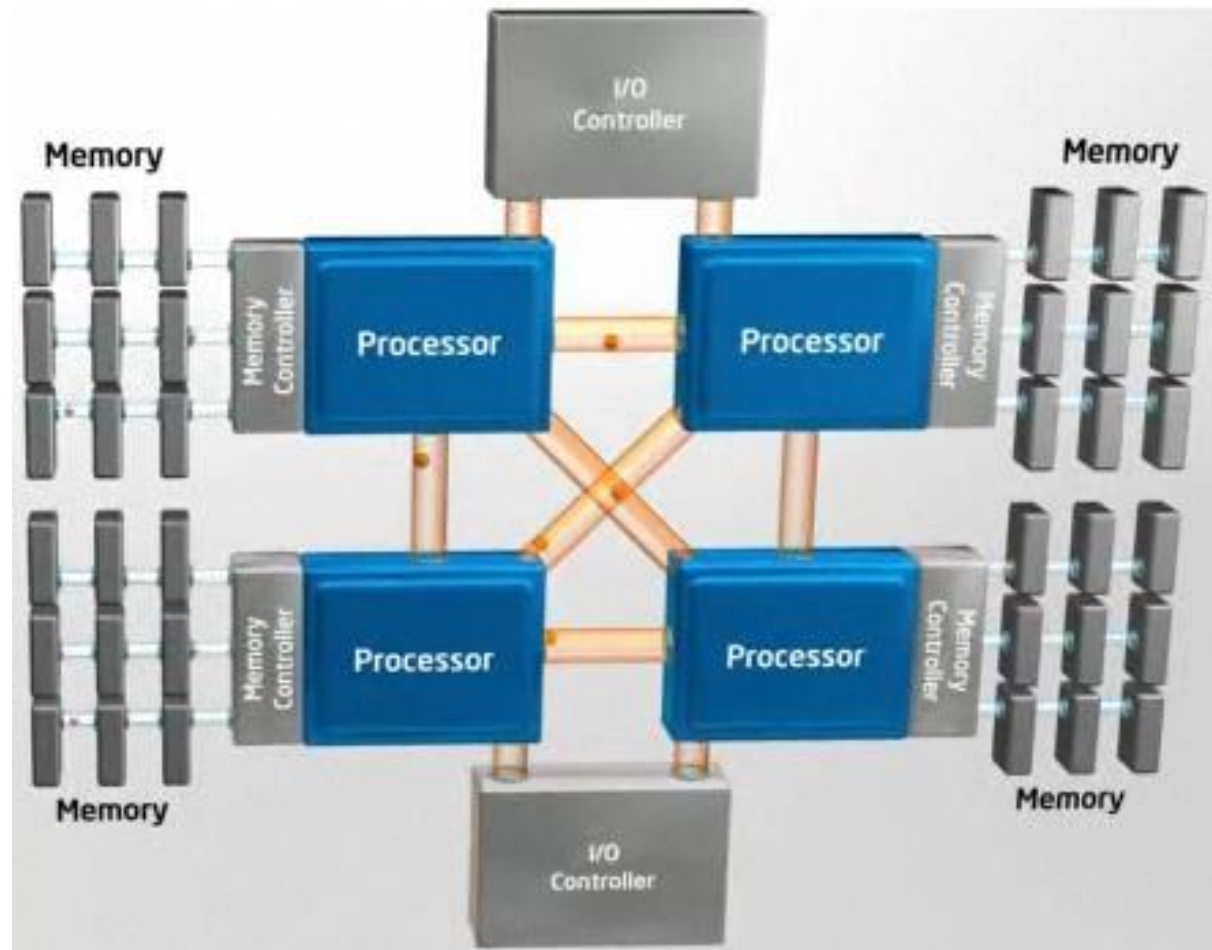
Intel Core 2 Duo in PC



Intel Core 2 Quad in PC

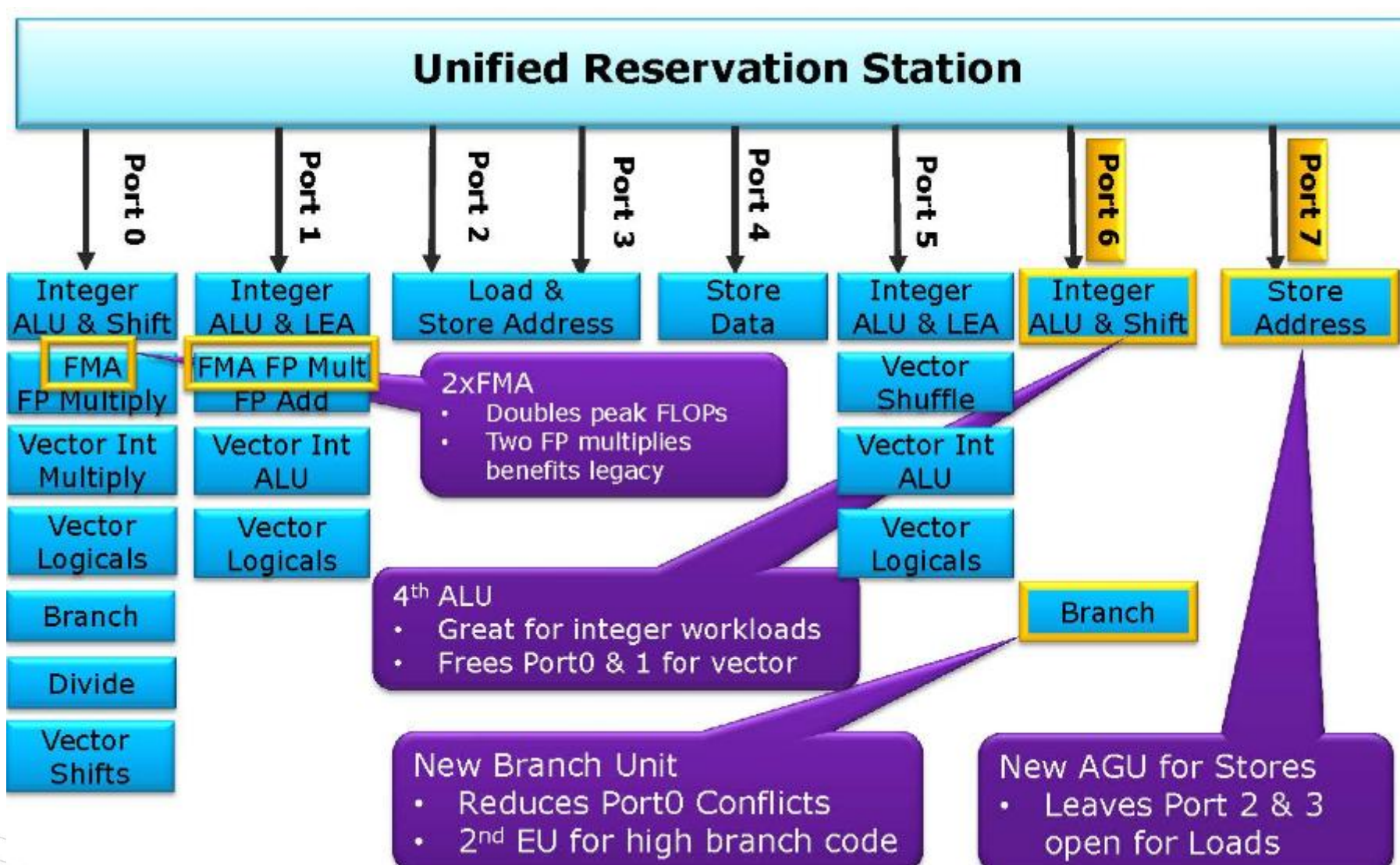


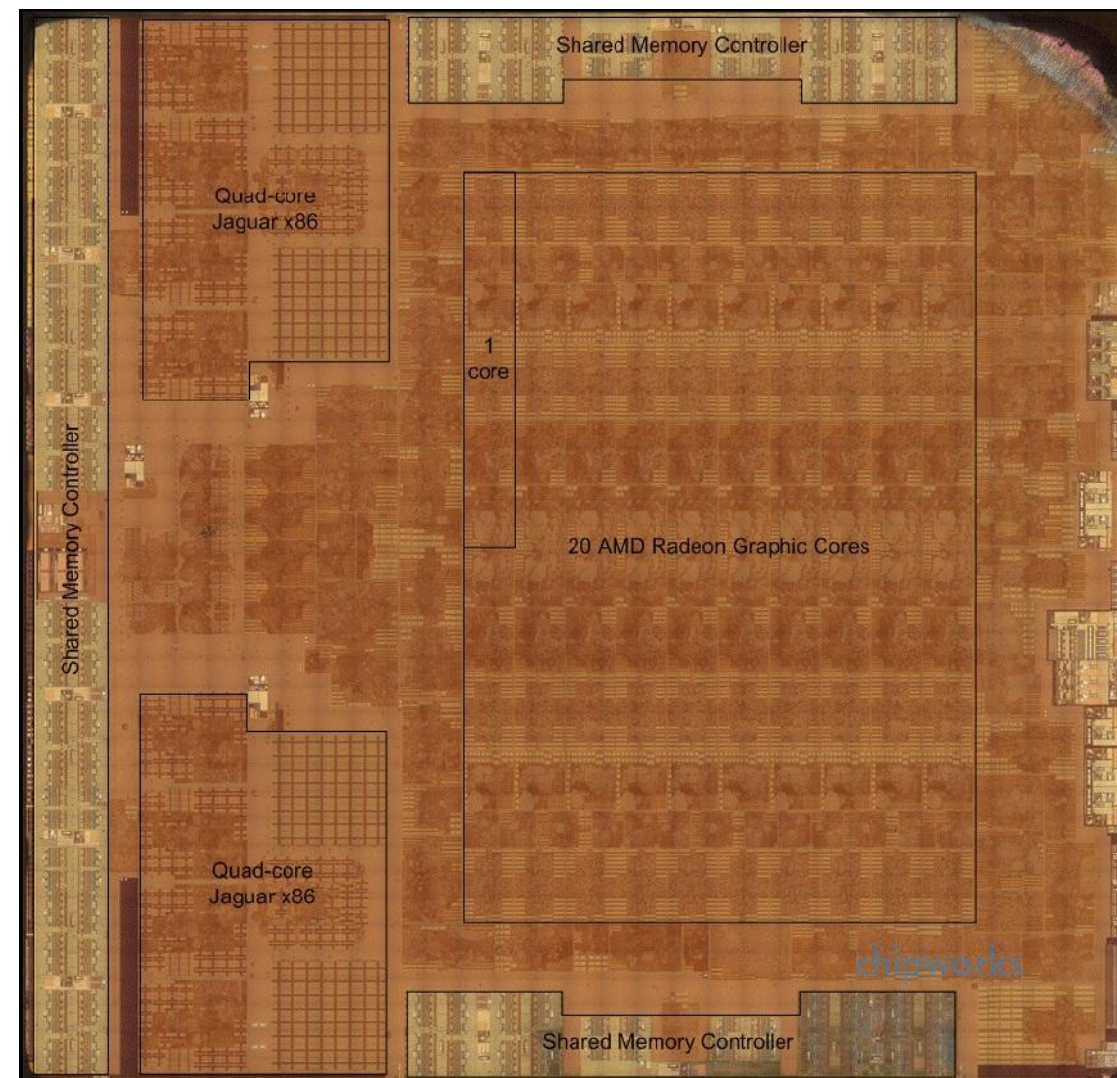
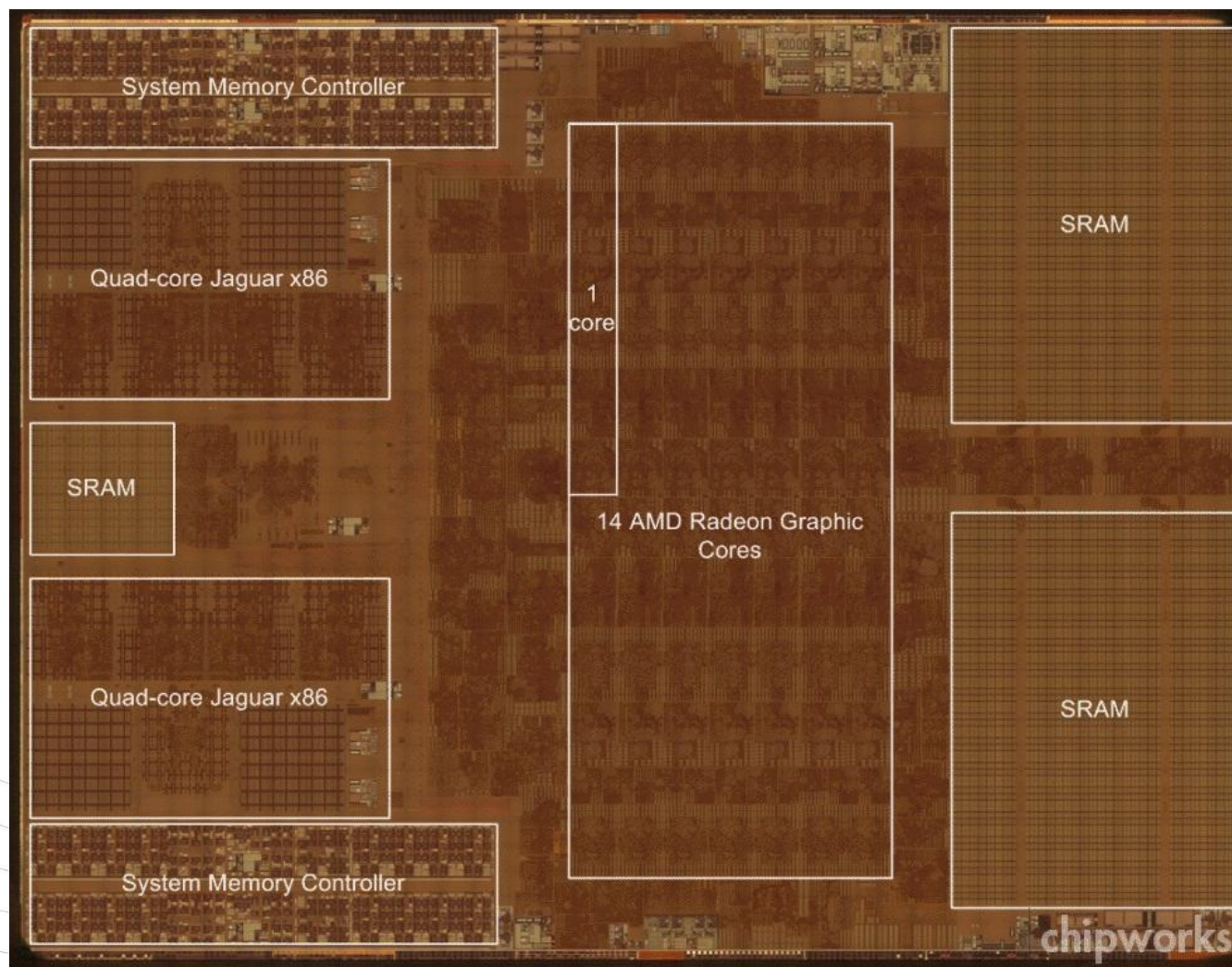
Intel Core i7



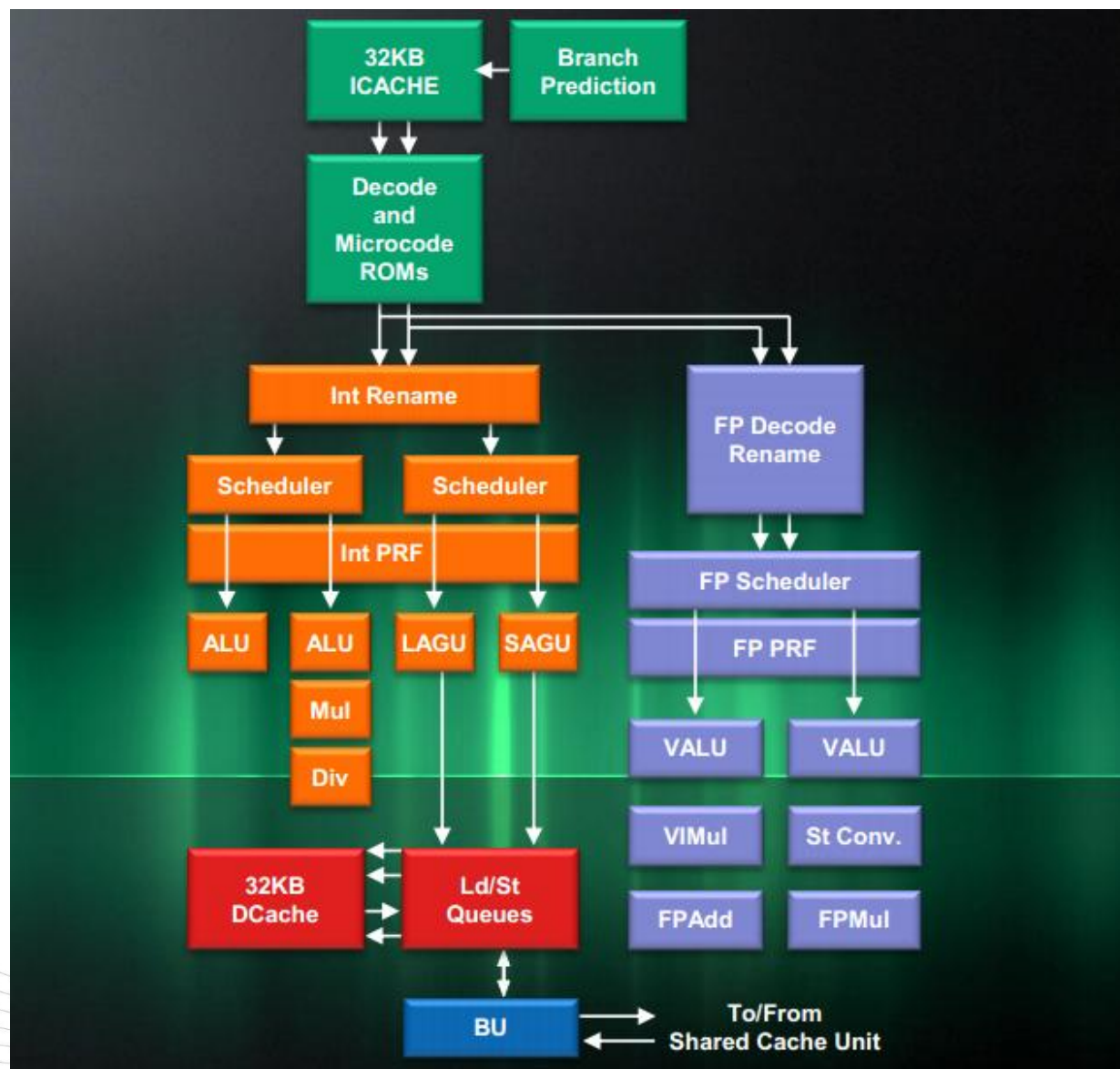


Core i7 Haswell



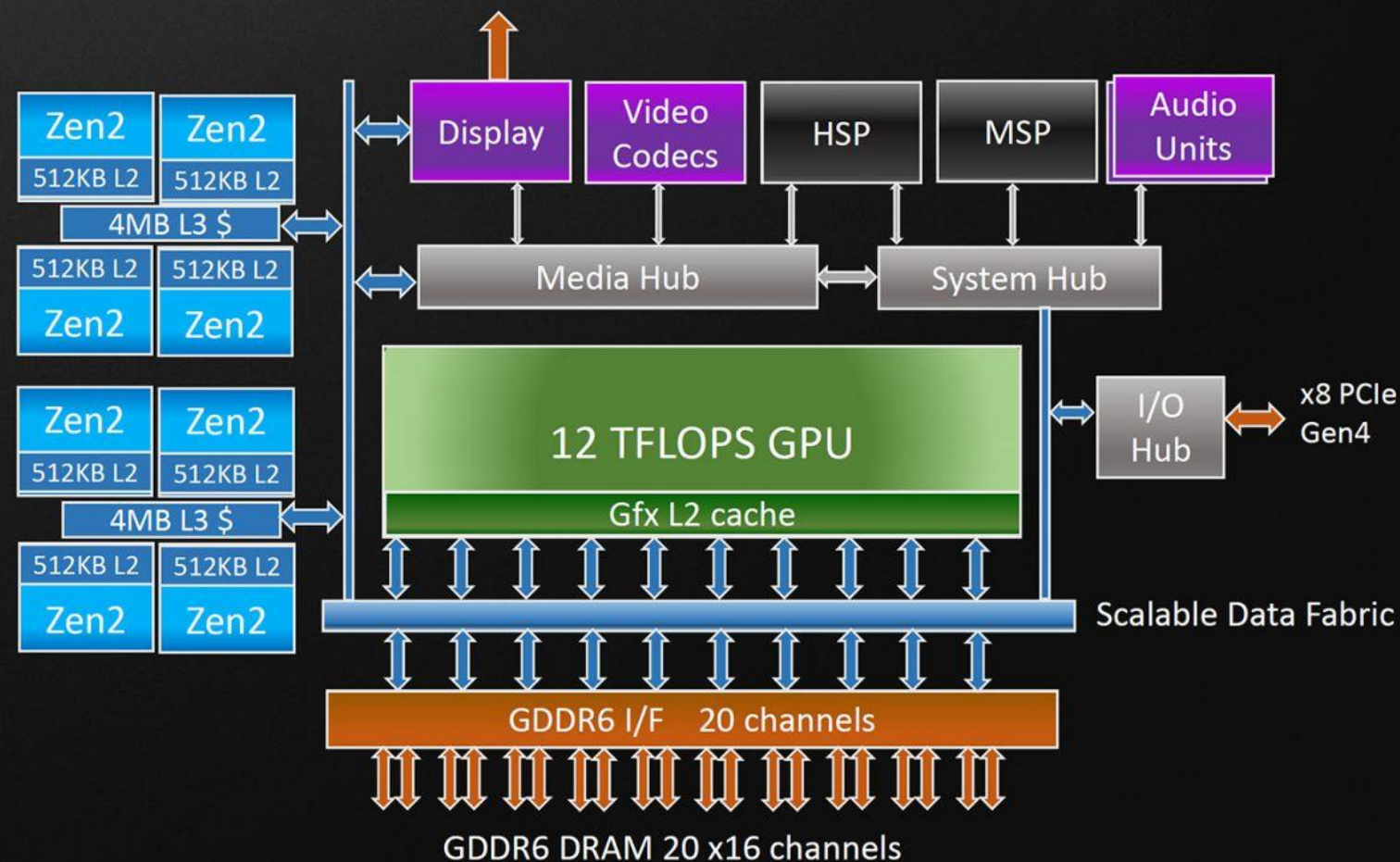


AMD x86 Jaguar

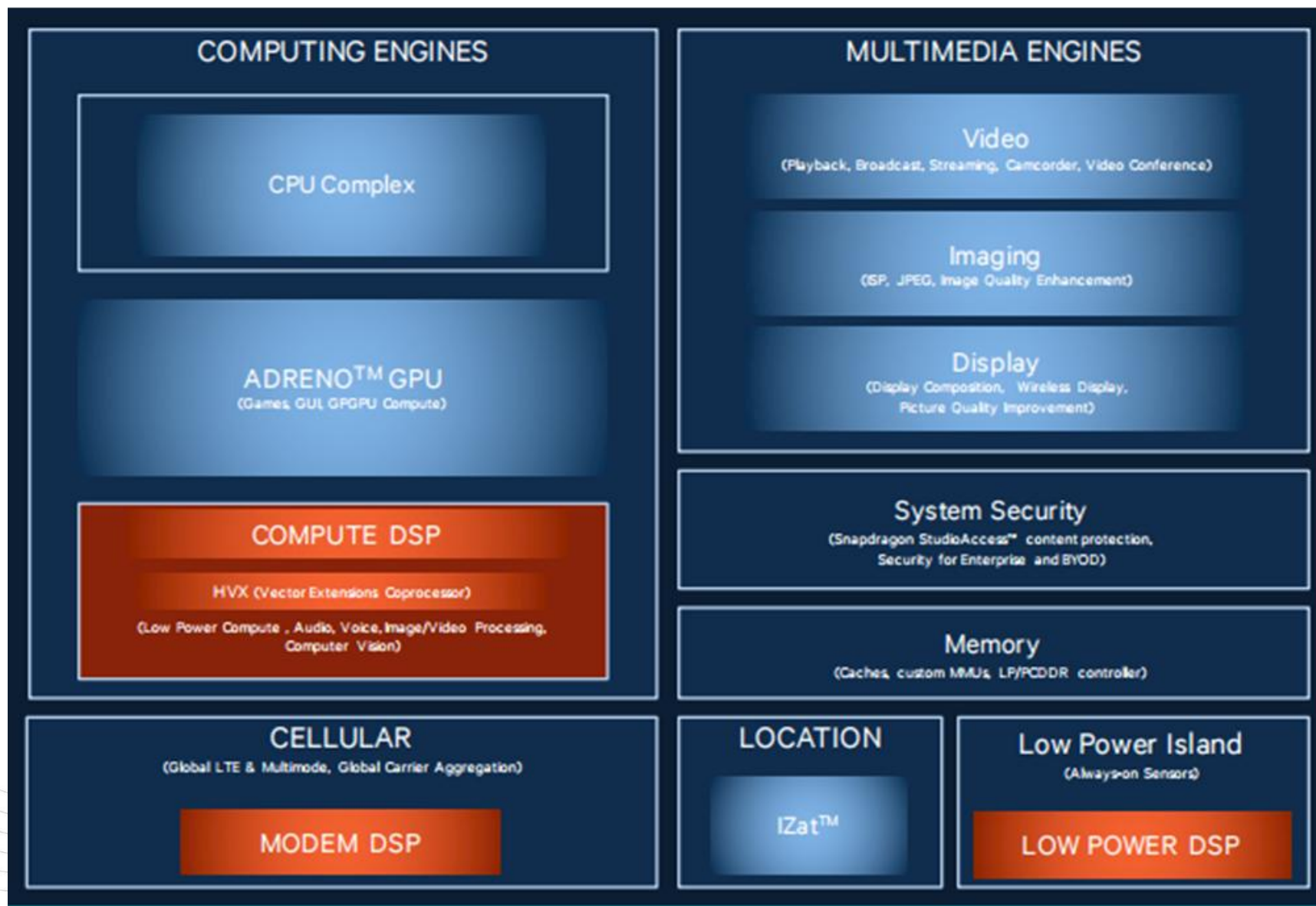


Xbox Series X

Xbox Series X | SOC Block Diagram



Snapdragon 820



Concurrency and Compilers

- Fundamental turning point in software development
- Compilers can no longer ensure that a piece of software makes ‘full’ use of the hardware performance
- Concurrency is required to make effective use of multi-core processors

Characteristics of Concurrency

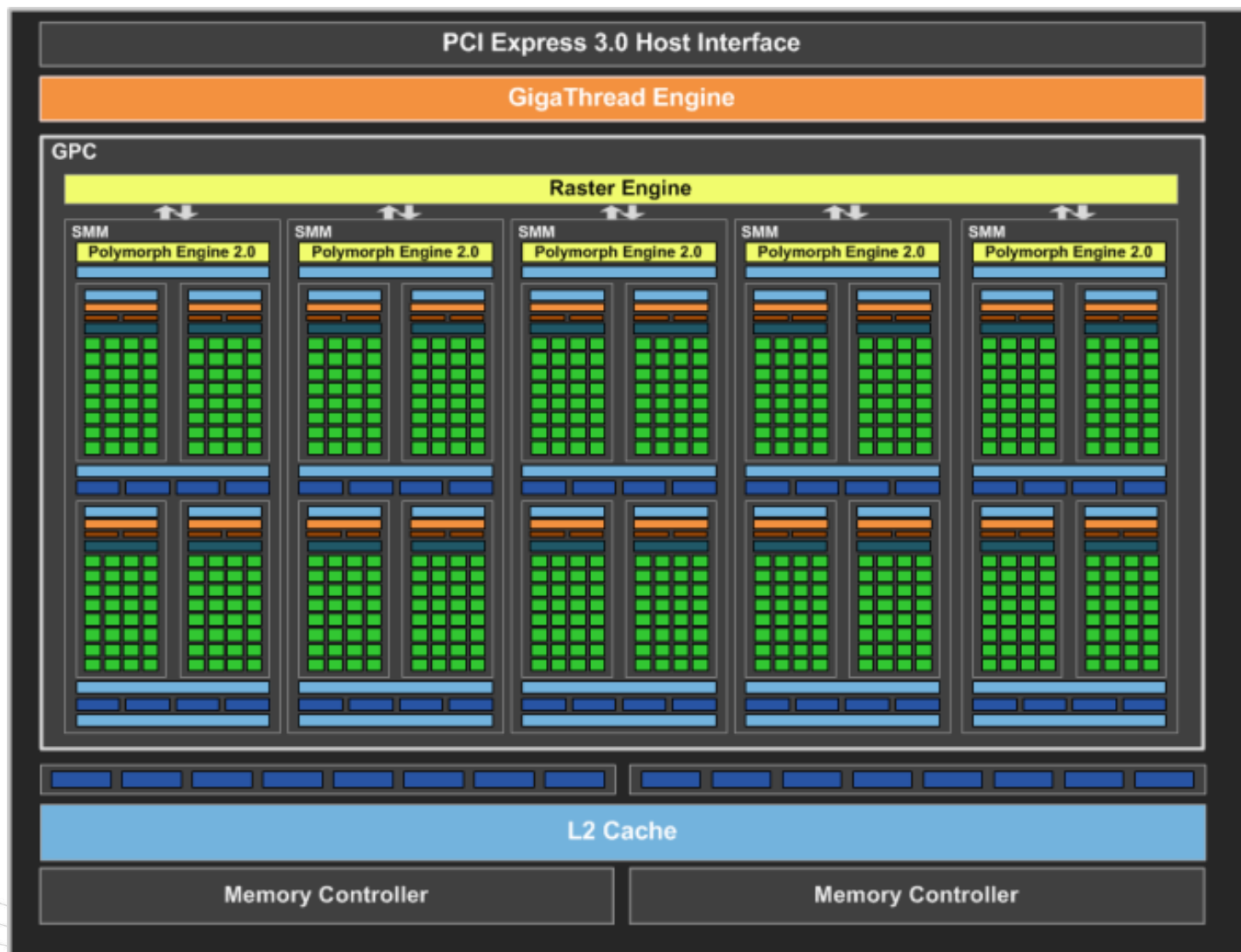
- Difficult to programme
- Difficult to debug
- Difficult to optimise
- Just hardVERY HARD

- Why ??

Process taxonomy

- Single Instruction Single Data: SISD
 - Standard processor
- Single Instruction Multiple Data: SIMD
 - Requires vector processor
- Single Program Multiple Data: SPMD
 - The same tasks run simultaneously on multiple processors
- Multiple Program Multiple Data: MPMD
 - Different tasks run simultaneously on multiple processors

GPU – NVIDIA Maxwell



GPU – NVIDIA Volta



Xbox Series X - GPU

Xbox Series X | GPU diagram

- 26 active Dual Compute Units
- Unified Geometry Engine
- Mesh Shading Geometry Engine
- Distributed Primitives & Rasterization
- Screen Tiled Color/Depth unit
- Multi-core Command Processor
- Three-level Cache, TLB



Xbox Series X – Compute Unit

Xbox Series X | Dual Compute Unit



- Four SIMDs plus 4 Scalar ALUs
- 32 Scalar FP32 FMAD per SIMD, 128 per Dual CU with data sharing
- Launch 7 instructions/clock per CU
 - 2 Vector ALU, 1 Vector Data, 2 Scalar, 2 Control
- 4 Texture or Ray ops/clock per CU
- Total per clock: 256 32b FP, 512 16b FP

Set Reading – Historical context

- The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software, By Herb Sutter
 - <http://www.gotw.ca/publications/concurrency-ddj.htm>
- Software and the Concurrency Revolution, Herb Sutter and James Larus, ACM Queue vol. 3, no. 7 - September 2005
 - <http://queue.acm.org/detail.cfm?id=1095421>

Principles of threading

Overview, priorities, scheduling and life cycle

Multithreading

- Thread
 - An independent unit of program execution
- Multithreaded program
 - Multiple threads execute simultaneously
 - Threads multiplexed onto available CPU's using time-slicing
 - Context switch incurs (minor) overhead
 - Threads exist in the same process and share common address space

Multithreading benefits on a single core architecture

- Improves performance only when there is an operation with some latency
 - longer the delay, the greater the potential performance benefit from using threads
- No improvement in performance if threads are competing for finite resources (e.g. CPU)
 - multithreading may result in worse performance

Multithreading benefits on a multi core architecture

- Improves performance irrespective of operation latency, by making use of idle cores.
 - The more cores the greater the potential speed increase.
- No improvement in performance if threads are competing for finite resources (e.g. memory)
 - multithreading may actually result in worse performance

Multithreading Issues

- Multithreading will introduce
 - additional complexity
 - unusual errors that are difficult to detect, replicate and debug
 - additional overhead; OS must manage all the threads
- Getting threading right is difficult; need to
 - use synchronization and locking to ensure correct interleaving of thread operations and interactions
 - balance potential improvements from threading against additional overheads (number of threads)
 - avoid deadlock, livelock, race conditions, etc caused by incorrect / insufficient / too much synchronization

Threads and Processes

- OS runs each application as a separate process
 - Increases the integrity/reliability of the system
- Process contains
 - Memory (Heap)
 - Image (Code)
 - One or more threads
- Threads are owned by a single process
- Threads are created and destroyed by the process
 - Threads can also destroy themselves
- The first thread (primary thread) is created by the OS when the process is started and the application loaded

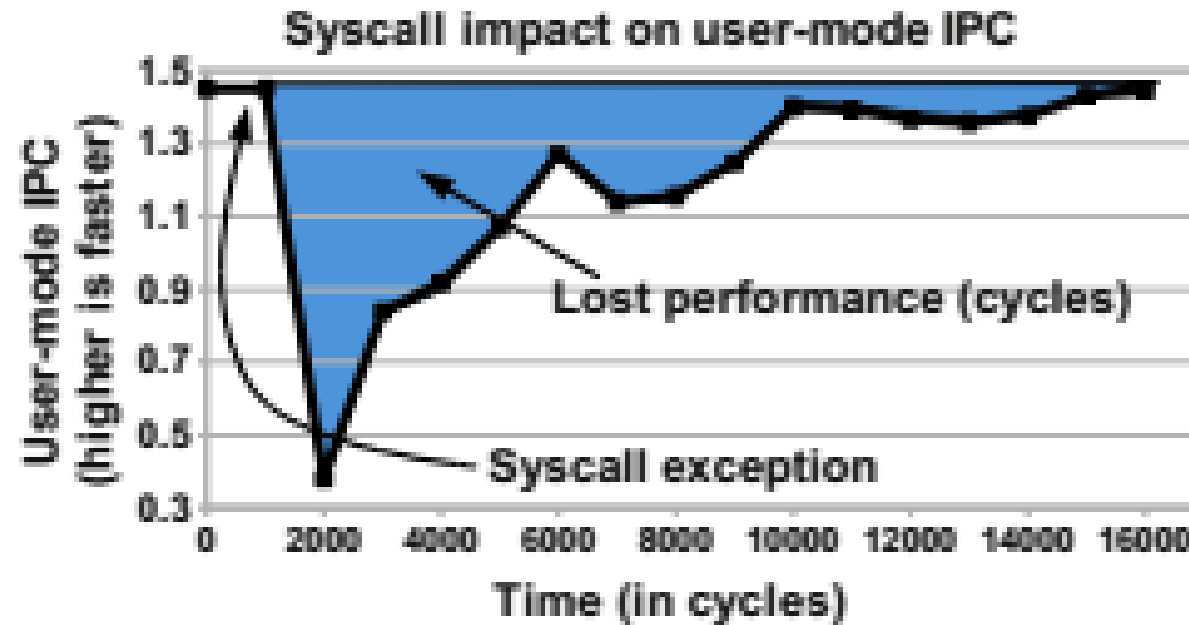
Threads context

- All threads have an associated thread context, containing:
 - User mode stack – temporary variables
 - Return addresses – function calls
 - Kernel mode stack – interrupts
 - Processor registers
- Switching context requires that the state of the CPU/thread be completely preserved, including:
 - Pipeline flushes
 - Cache flushes
- A context is switched when the scheduler stops one thread and replaces it with another
- Context switching occurs hundreds of times a second
- Represents an overhead

Thread priorities

- Scheduler assigns threads to the CPU on the basis of thread priority
 - Process priority
 - Real-time, High, Normal, Idle
 - Thread priority
 - Time critical, Highest, Above Normal, Normal, Below Normal, Lowest, Idle
- Only increase the priority of a thread when more CPU time is required, and such time is likely to be available
 - Giving a thread too high a priority can lead to starvation of other threads
- Threads are interrupted and scheduled (swapped) out when
 - A more important thread is required to run*
 - The threads current time slot expired
 - The thread suspends itself
 - The thread terminates

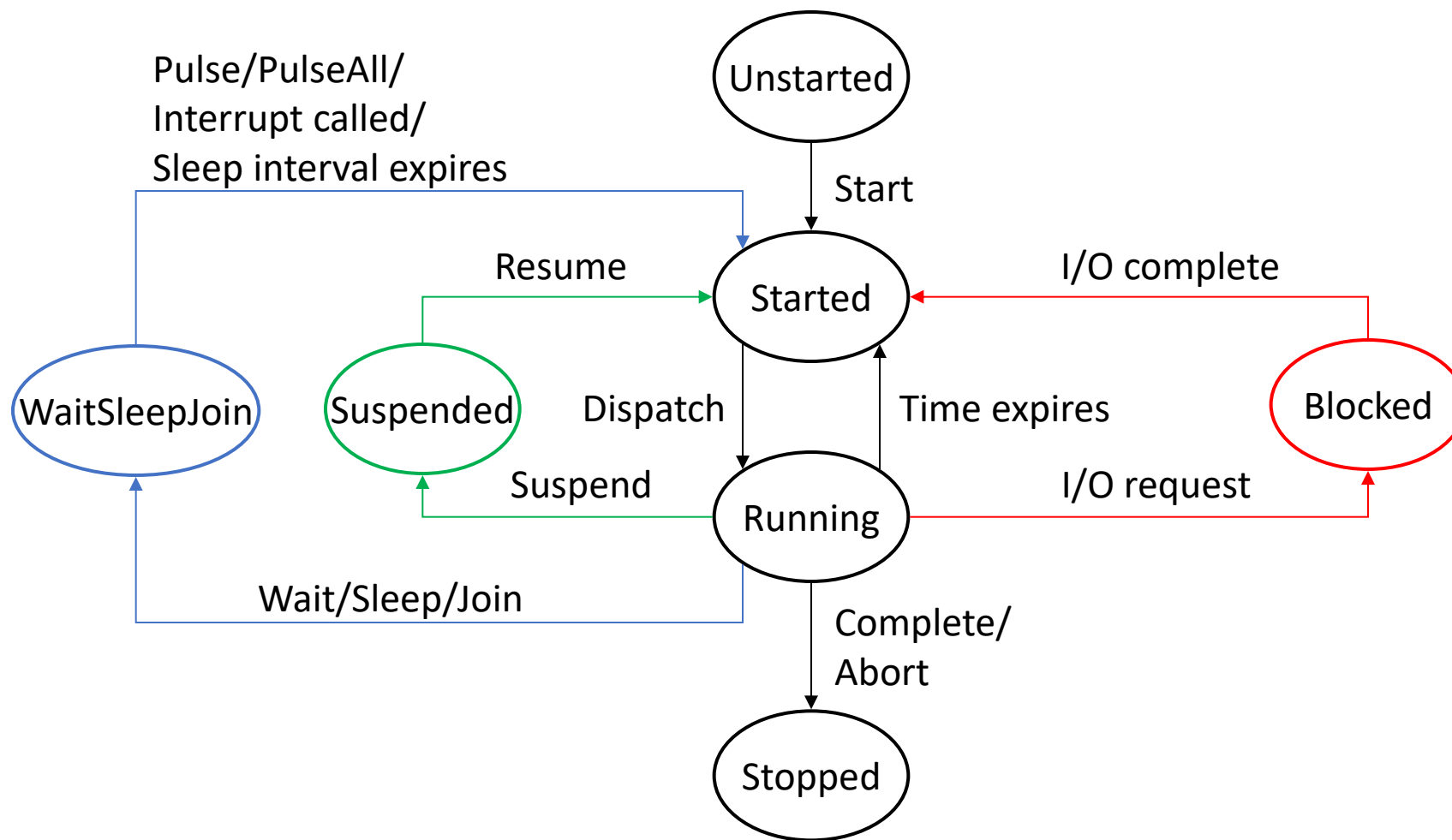
The cost of scheduling



Reference

http://www.cs.cmu.edu/~chensm/Big_Data_reading_group/papers/flexsc-osdi10.pdf

Thread life cycle



Common Terms

- Create
 - The thread comes into existence
- Suspend
 - The thread is stopped
- Release
 - The suspended thread is started
- Sleep
 - Suspending the thread
- Terminate
 - The thread ceases to exist

The reality

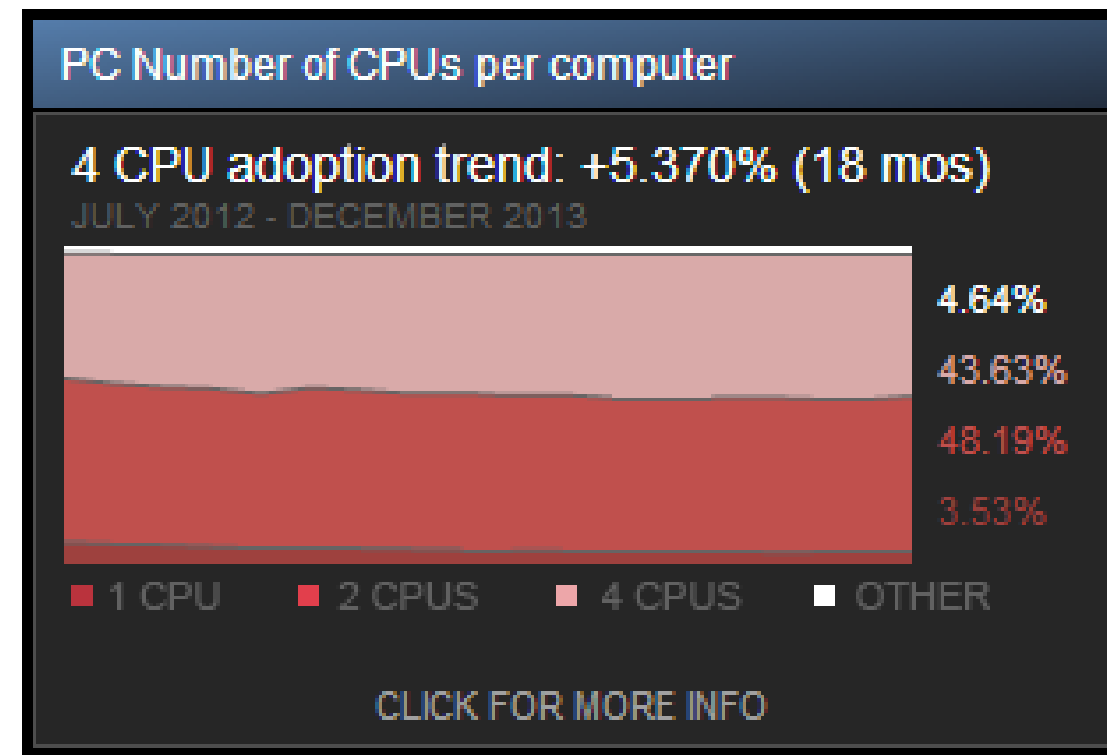
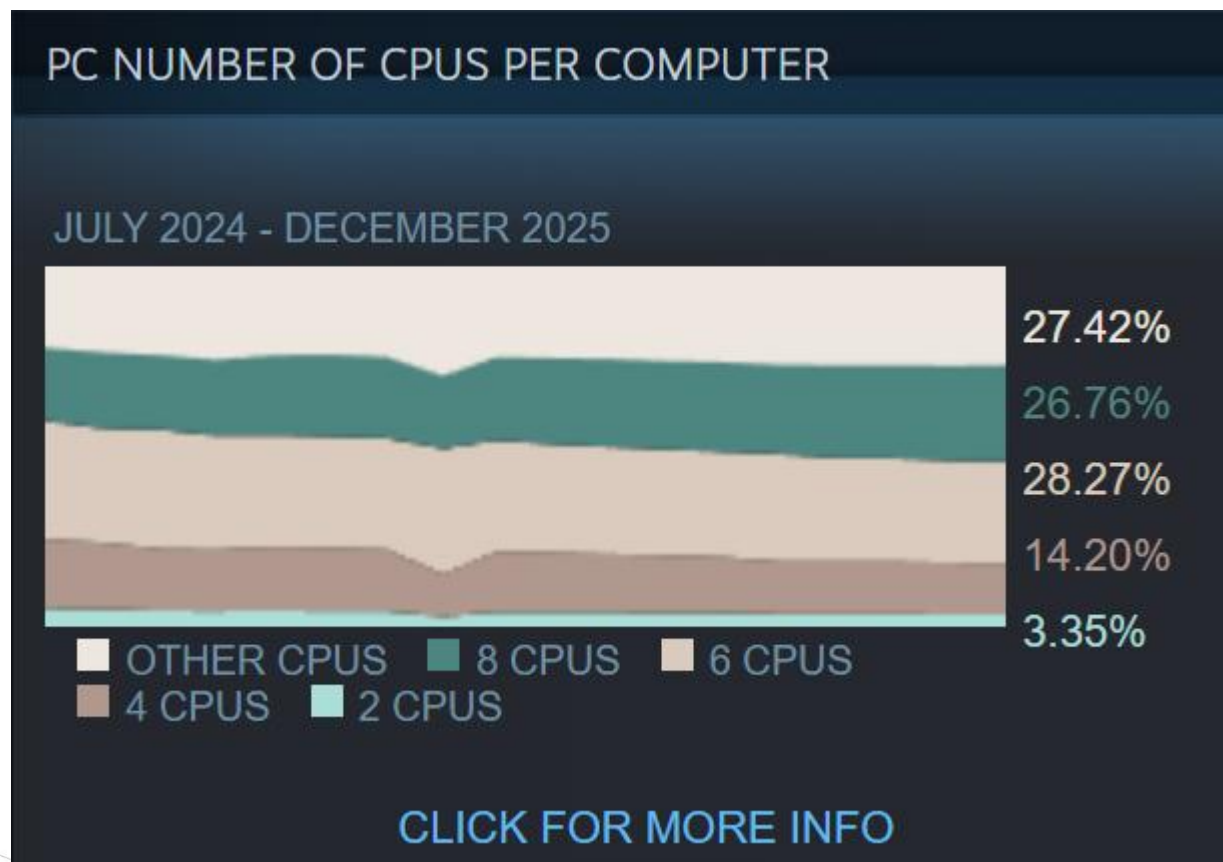
- The current generation hardware is multi-core
 - PCs
 - Game consoles (Xbox360/XboxOne/Xbox X series/PS3/PS4/PS5)
- The next generation of hardware will be massively multi-cored
- Maximum performance will only be available if the application is multithreaded

Development options - CPU

- Ignore it: write unthreaded code
- **MT-enabled language** (e.g. C#, Python, C++ 11)
- MT middleware
- Cross platform MT libraries (e.g. Pthreads)
- **OS-specific MT libraries** (e.g. Win32)
- Lock-free programming
- **OpenMP**
- MPI
- **GPGPU** (e.g. OpenCL, CUDA, Compute Shader)

Steam hardware survey

Dec 2025

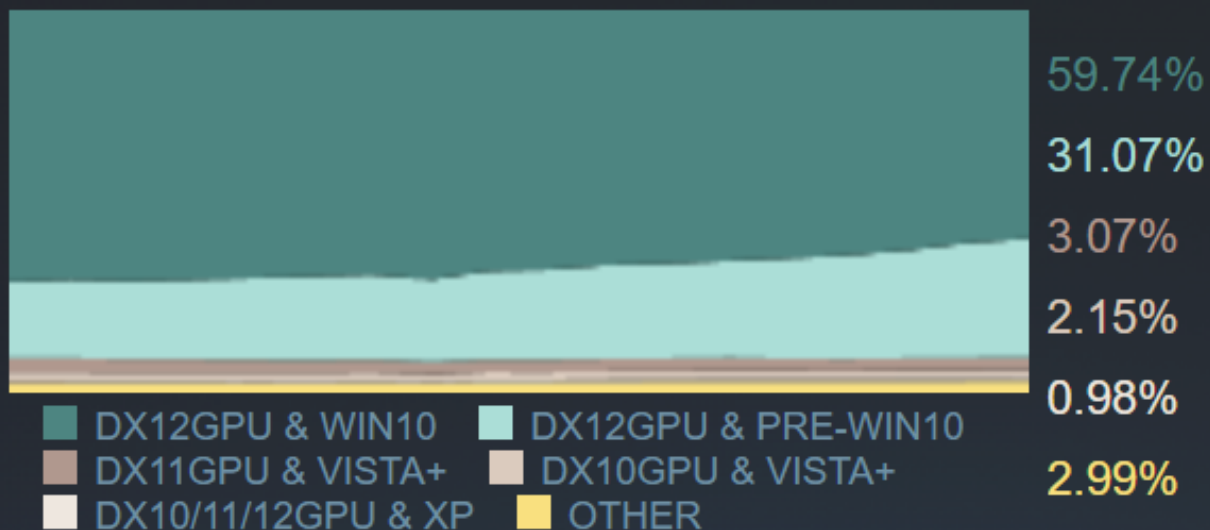


Steam survey

Info only

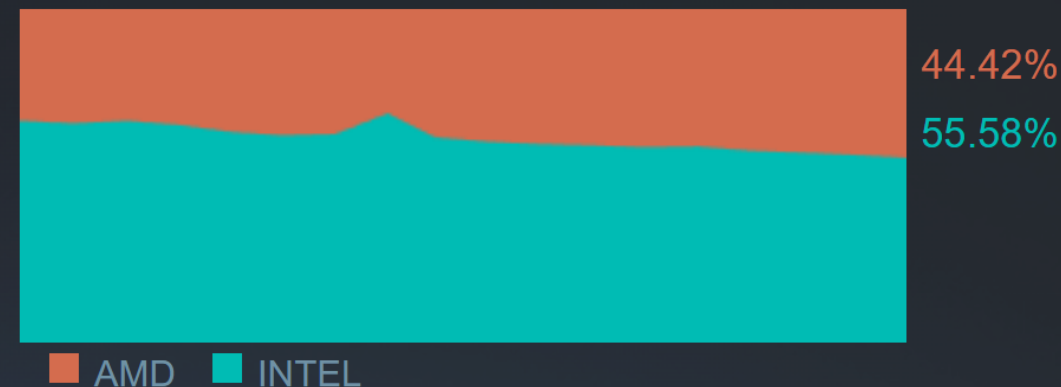
DX10/11/12 SYSTEMS

JULY 2024 - DECEMBER 2025



PC PROCESSOR USAGE BY MANUFACTURER

JULY 2024 - DECEMBER 2025



PC VIDEO CARD USAGE BY MFG

JULY 2024 - DECEMBER 2025

